

CAD 种子填充插件开发实现思路文档

目录

- 一、插件使用说明3
 - 1.1 环境要求 3
 - 1.2 加载插件 3
 - 1.3 使用步骤 3
 - 1.4 命令说明 4
- 二、整体架构设计4
 - 2.1 模块划分 4
 - 2.2 类继承关系 4
 - 2.3 数据流 4
- 三、输入校验模块5
 - 3.1 种子校验 (ValidateSeed) 5
 - 3.2 边界校验 (ValidateBoundary) 5
- 四、网格全覆盖算法 (核心)5
 - 4.1 问题分析 5
 - 4.2 方向向量定义 6
 - 4.3 行列范围计算 6
 - 4.4 网格行列推导过程 6
 - 4.5 网格坐标计算核心代码 8
- 五、瓦片分类判断8
 - 5.1 完全在外 (IsCompletelyOutside) 8
 - 5.2 完全在内 (IsFullyInside) 9
 - 5.3 射线法 (IsPointInsidePolyline) 10
- 六、边界裁剪 11
 - 6.1 方案探索与决策 11
 - 6.2 最终方案：边修剪 + 凸包算法 11
- 七、统计功能 13
- 八、图层管理 14

九、关键技术难点与解决..... 14

十、总结与展望..... 15

一、插件使用说明

1.1 环境要求

- AutoCAD 2022
- Windows 10/11 64 位
- .NET Framework 4.8

1.2 加载插件

1. 启动 AutoCAD 2022
2. 在命令行输入 `NETLOAD`，回车
3. 选择 项目存放位置\BIMSeedFiller_交付\源码\BIMSeedFiller\BIMSeedFiller\bin\Debug 里的`BIMSeedFiller.dll`，点击打开

1.3 使用步骤

1. 用`PLINE`命令绘制闭合边界，选中后在图层下拉菜单中赋予"边界"图层
2. 用`RECTANG`或`PLINE`命令绘制矩形种子，选中后在图层下拉菜单中赋予"种子"图层（种子必须完全位于边界内部，且为闭合矩形）
3. 在命令行输入 `Seed`，回车
4. 按提示依次选择种子和边界
5. 等待程序自动完成填充
6. 在命令行输入`Statistics`，回车
7. 选择边界，查看统计结果

1.4 命令说明

命令	功能
Seed	选择种子和边界，自动铺满整个边界内部区域
statistics	选择一个已填充的边界，统计完整种子和裁切种子的个数及面积

二、整体架构设计

2.1 模块划分

本插件采用经典的分层架构，遵循单一职责原则：

模块	文件	职责
命令入口	Commands.cs	插件命令入口，负责调度，不包含业务逻辑
输入校验	InputCollector.cs	用户输入获取与校验，确保输入数据合法性
核心算法	TileFiller.cs	网格生成、内外判断、裁剪、统计
数据模型	Tile.cs、 StandardTile.cs、 ClippedTile.cs	OOP 继承体系
数据容器	SeedData.cs、 BoundaryData.cs	输入数据的结构化容器
统计计算	TileStatistics.cs	对填充结果进行分类统计

2.2 类继承关系

Tile (抽象基类)

└── StandardTile (完整矩形瓦片，有扩散能力)

└── ClippedTile (被裁剪过的瓦片)

2.3 数据流

用户输入 → InputCollector（校验） → SeedData / BoundaryData → TileFiller（填充） → Dictionary<ObjectId, List<Tile>> allResults → Statistics（统计输出）

三、输入校验模块

3.1 种子校验 (ValidateSeed)

对用户选择的种子矩形进行 8 项严格校验：

1. 检查能否正确获得种子即 obj 不为空"
2. 必须为 `Polyline` 类型
3. 检查图层名是不是 "种子"
4. 检查是否闭合：Polyline.Closed == true?
5. 检查是否全由直线构成：遍历所有顶点，确保每个顶点的 Bulge 值都为 0。
6. 检查顶点数是不是 4
7. 检查是否为矩形（点积接近 0）
8. 面积必须大于 0

3.2 边界校验 (ValidateBoundary)

对用户选择的边界进行 5 项严格校验：

1. 检查能否正确获得边界
2. 检查是不是 Polyline
3. 检查图层名是不是 "边界"
4. 检查是否闭合：Polyline.Closed == true?
5. 边界的面积必须大于 0

四、网格全覆盖算法（核心）

4.1 问题分析

种子可能带有旋转角度，边界可能包含圆弧段。传统的 BFS 扩散方案在旋转种子上存在去重困难。本方案采用"旋转网格全覆盖 + 统一裁剪"的两阶段策略。

4.2 方向向量定义

设种子宽度为 `w`，高度为 `h`，旋转角度为 `θ`：

- 列方向（右邻居）： $u = (\cos\theta \cdot w, \sin\theta \cdot w)$

- 行方向（上邻居）： $v = (-\sin\theta \cdot h, \cos\theta \cdot h)$

任意瓦片中心点坐标： $P(i, j) = \text{Center} + i \cdot u + j \cdot v$

4.3 行列范围计算

为保证网格完全覆盖边界包围盒，取包围盒的 8 个采样点（4 角点 + 4 边中点），对每个采样点反算其在旋转网格中的浮点行列号：

$$i = ((P.X - C.X) \cdot v.Y - (P.Y - C.Y) \cdot v.X) / (w \cdot h)$$

$$j = ((P.Y - C.Y) \cdot u.X - (P.X - C.X) \cdot u.Y) / (w \cdot h)$$

分母 `w·h` 恒为正，无除零风险。取所有采样点的最小/最大整数行列号，向外扩展一格，保证绝对覆盖。

4.4 网格行列推导过程

以下推导过程为手写稿

i, j 分别为行号与列号
 seed.center 种子中心点
 u, v 方向向量

设 $P = (P.x, P.y)$ $\text{seed.center} = (C.x, C.y)$
 $u = (u.x, u.y)$ $v = (v.x, v.y)$
 $P = \text{seed.center} + i \times u + j \times v$
 $\Rightarrow \begin{cases} P.x = C.x + i \times u.x + j \times v.x \\ P.y = C.y + i \times u.y + j \times v.y \end{cases}$
 $\Rightarrow \begin{cases} P.x - C.x = i \times u.x + j \times v.x \\ P.y - C.y = i \times u.y + j \times v.y \end{cases}$

$$\frac{1}{2} P \cdot x - C \cdot x = dx \quad P \cdot y - C \cdot y = dy$$

$$\text{得} \begin{cases} i \cdot u \cdot x + j \cdot v \cdot x = dx \\ i \cdot u \cdot y + j \cdot v \cdot y = dy \end{cases}$$

$$\Rightarrow \begin{cases} i = \frac{dx \cdot xv \cdot y - dy \cdot xv \cdot x}{u \cdot x \cdot xv \cdot y - u \cdot y \cdot xv \cdot x} \\ j = \frac{dy \cdot u \cdot x - dx \cdot u \cdot y}{u \cdot x \cdot xv \cdot y - u \cdot y \cdot xv \cdot x} \end{cases}$$

$$u \cdot x \cdot xv \cdot y - u \cdot y \cdot xv \cdot x$$

$$\Rightarrow \cos \theta \cdot w \times \cos \theta \cdot h - \sin \theta \cdot w \times - \sin \theta \cdot h$$

$$= w \cdot h \cdot \cos (\cos^2 \theta + \sin^2 \theta)$$

$$= w \cdot h$$

4.5 网格坐标计算核心代码

```
// 4.3 计算瓦片的行和列的最大值和最小值
// 4.3.1 计算方向向量
double cos = Math.Cos(seed.Rotation);
double sin = Math.Sin(seed.Rotation);
Vector3d u = new Vector3d(cos * seed.Width, sin * seed.Width, 0); // 列方向
Vector3d v = new Vector3d(-sin * seed.Height, cos * seed.Height, 0); // 行方向
double det = seed.Width * seed.Height; // 行列式, 不等于0

// 4.3.2 遍历采样点然后找出最大行列和最小行列
int minCol = int.MaxValue;
int maxCol = int.MinValue;
int minRow = int.MaxValue;
int maxRow = int.MinValue;

foreach (Point3d pt in samplePoints)
{
    double offsetX = pt.X - seed.Center.X;
    double offsetY = pt.Y - seed.Center.Y;
    double a = (offsetX * v.Y - offsetY * v.X) / det; // 列号
    double b = (offsetY * u.X - offsetX * u.Y) / det; // 行号

    int col = (int)Math.Floor(a);
    int row = (int)Math.Floor(b);
    if (col < minCol) minCol = col;
    if (col > maxCol) maxCol = col;
    if (row < minRow) minRow = row;
    if (row > maxRow) maxRow = row;
}

// 4.3.3 再扩大一圈, 做保险
minCol--;
minRow--;
maxCol++;
maxRow++;
```

五、瓦片分类判断

5.1 完全在外 (IsCompletelyOutside)

三道筛选, 逐层排除:

1. 包围盒排斥: 比较瓦片与边界的包围盒, 若完全分离则必然在外
2. 边相交检测: 检查瓦片四条边是否与边界有交点, 有交点则说明相交
3. 顶点位置判定: 取瓦片一个顶点, 用射线法判断是否在边界内


```

/// <summary>
/// 判断瓦片是否完全处于边界外围
/// </summary>
/// <param name="rect"></param>
/// <param name="boundaryGeom"></param>
/// <param name="boundaryBox"></param>
/// <returns></returns>
1 个引用 | Gpt
private static bool IsCompletelyOutside(Polyline rect, Polyline boundaryGeom, Extents3d boundaryBox)
{
    // 1. 包围盒排序
    Extents3d a = rect.GeometricExtents;
    if (a.MinPoint.X > boundaryBox.MaxPoint.X || a.MaxPoint.X < boundaryBox.MinPoint.X ||
        a.MinPoint.Y > boundaryBox.MaxPoint.Y || a.MaxPoint.Y < boundaryBox.MinPoint.Y)
    {
        return true;
    }

    // 2. 边相交检测
    for (int i = 0; i < 4; i++)
    {
        Point3d p1 = rect.GetPoint3dAt(i);
        Point3d p2 = rect.GetPoint3dAt((i + 1) % 4);
        Point3dCollection pts = new Point3dCollection();
        Line tempLine = new Line(p1, p2);
        boundaryGeom.IntersectWith(tempLine, Intersect.OnBothOperands, pts, IntPtr.Zero, IntPtr.Zero);
        tempLine.Dispose();
        if (pts.Count > 0)
            return false;
    }

    // 3. 顶点位置判定
    if (IsPointInsidePolyline(rect.GetPoint3dAt(0), boundaryGeom))
        return false;

    return true;
}

```

5.2 完全在内 (IsFullyInside)

1. 四个顶点必须全部在边界内部
2. 四条边必须全部与边界无交点（防止凹角误判——四个顶点都在内但瓦片边缘跨出边界）

```

/// <summary>
/// 判断瓦片是否完全处于边界内部
/// </summary>
/// <param name="rect"></param>
/// <param name="boundaryGeom"></param>
/// <returns></returns>
1 个引用 | Gpt
private static bool IsFullyInside(Polyline rect, Polyline boundaryGeom)
{
    // 先检查四个顶点是否都在内部
    for (int i = 0; i < 4; i++)
    {
        if (!IsPointInsidePolyline(rect.GetPoint3dAt(i), boundaryGeom))
            return false;
    }

    // 再检查四条边是否与边界有交点（防止凹角误判）
    for (int i = 0; i < 4; i++)
    {
        Point3d p1 = rect.GetPoint3dAt(i);
        Point3d p2 = rect.GetPoint3dAt((i + 1) % 4);
        Point3dCollection pts = new Point3dCollection();
        Line tempLine = new Line(p1, p2);
        boundaryGeom.IntersectWith(tempLine, Intersect.OnBothOperands, pts, IntPtr.Zero, IntPtr.Zero);
        tempLine.Dispose();
        if (pts.Count > 0)
            return false; // 有交点，说明瓦片跨越了边界
    }

    return true; // 顶点全在内，且没有边与边界相交，才是真正的完全内部
}

```

5.3 射线法 (IsPointInsidePolyline)

从点向右发水平射线，统计与边界的交点数量。奇数在内，偶数在外。对圆弧段采用离散化处理（10段直线近似）。

```
/// <summary>
/// 判断顶点位置是否完全在边界内部
/// </summary>
/// <param name="point"></param>
/// <param name="boundaryGeom"></param>
/// <returns></returns>
3 个引用 | Gpt
private static bool IsPointInsidePolyline(Point3d point, Polyline boundaryGeom)
{
    if (!boundaryGeom.Closed) return false;

    Point2d pt = new Point2d(point.X, point.Y);
    bool inside = false;
    int n = boundaryGeom.NumberOfVertices;

    for (int i = 0, j = n - 1; i < n; j = i++)
    {
        // 直线段
        if (boundaryGeom.GetBulgeAt(j) == 0)
        {
            Point2d p1 = boundaryGeom.GetPoint2dAt(j);
            Point2d p2 = boundaryGeom.GetPoint2dAt(i);

            if ((p1.Y > pt.Y) != (p2.Y > pt.Y) &&
                pt.X < (p2.X - p1.X) * (pt.Y - p1.Y) / (p2.Y - p1.Y) + p1.X)
            {
                inside = !inside;
            }
        }
        // 圆弧段：离散为10段直线进行判断
        else
        {
```

```
            else
            {
                double bulge = boundaryGeom.GetBulgeAt(j);
                Point2d p1 = boundaryGeom.GetPoint2dAt(j);
                Point2d p2 = boundaryGeom.GetPoint2dAt(i);

                double chordLen = p1.GetDistanceTo(p2);
                double theta = Math.Atan2(Math.Abs(bulge), 1) * 4 * Math.Sign(bulge);
                double radius = chordLen / (2 * Math.Sin(Math.Abs(theta) / 2));
                if (radius <= 0) continue;

                double midX = (p1.X + p2.X) / 2, midY = (p1.Y + p2.Y) / 2;
                double dist = Math.Sqrt(Math.Max(0, radius * radius - (chordLen / 2) * (chordLen / 2)));
                double midAngle = Math.Atan2(p2.Y - p1.Y, p2.X - p1.X) + (bulge > 0 ? Math.PI / 2 : -Math.PI / 2);
                double centerX = midX + dist * Math.Cos(midAngle);
                double centerY = midY + dist * Math.Sin(midAngle);

                double startAngle = Math.Atan2(p1.Y - centerY, p1.X - centerX);
                double endAngle = startAngle + theta;
                int samples = 10;

                for (int k = 0; k < samples; k++)
                {
                    double ang1 = startAngle + theta * k / samples;
                    double ang2 = startAngle + theta * (k + 1) / samples;
                    double sx = centerX + radius * Math.Cos(ang1);
                    double sy = centerY + radius * Math.Sin(ang1);
                    double ex = centerX + radius * Math.Cos(ang2);
                    double ey = centerY + radius * Math.Sin(ang2);

                    if ((sy > pt.Y) != (ey > pt.Y) &&
                        pt.X < (ex - sx) * (pt.Y - sy) / (ey - sy) + sx)
                    {
                        inside = !inside;
                    }
                }
            }
        }
    }
}
```

六、边界裁剪

6.1 方案探索与决策

先后尝试了三种裁剪方案：

方案	原理	结果
Region 布尔运算	AutoCAD 原生布尔运算	在当前项目环境中频繁出现对象释放异常
MPolygon	AutoCAD 多边形布尔类	需要额外引用 AcMPolygonMgd.dll, 环境中无法成功激活
手工点集重建	收集点→排序→建多边形	极角排序在凹多边形下连线异常

6.2 最终方案：边修剪 + 凸包算法

边修剪阶段：在矩形四条边上插入与边界的交点，通过判断每段中点是否在边界内，只保留内部的线段片段。

```
/// <summary>
/// 基于“边修剪”的裁剪算法 V3。独立处理每条边，只保留内部的线段片段，
/// 并使用凸包算法重建裁剪多边形，消除乱连线。
/// </summary>
/// <param name="tileRect"></param>
/// <param name="boundary"></param>
/// <param name="tr"></param>
/// <param name="db"></param>
/// <returns></returns>
1 个引用 | Gpt
private static List<Polyline> ComputeIntersection(Polyline tileRect, Polyline boundary,
    Transaction tr, Database db)
{
    List<Point3d> allInternalPoints = new List<Point3d>();

    // 1. 独立处理矩形的每条边，收集在边界内部的线段端点
    for (int i = 0; i < 4; i++)
    {
        Point3d start = tileRect.GetPoint3dAt(i);
        Point3d end = tileRect.GetPoint3dAt((i + 1) % 4);

        // 获取这条边上的交点
        Point3dCollection intPts = new Point3dCollection();
        using (Line tempLine = new Line(start, end))
            boundary.IntersectWith(tempLine, Intersect.OnBothOperands, intPts, IntPtr.Zero, IntPtr.Zero);

        // 构建边上的有序点序列
        List<Point3d> edgePoints = new List<Point3d> { start };
        List<Point3d> sortedInts = new List<Point3d>();
        foreach (Point3d pt in intPts) sortedInts.Add(pt);
        sortedInts.Sort((a, b) => start.DistanceTo(a).CompareTo(start.DistanceTo(b)));
        edgePoints.AddRange(sortedInts);
        edgePoints.Add(end);

        // 保留内部的线段片段
        for (int j = 0; j < edgePoints.Count - 1; j++)
        {
            Point3d mid = new Point3d(
                (edgePoints[j].X + edgePoints[j + 1].X) / 2,
                (edgePoints[j].Y + edgePoints[j + 1].Y) / 2, 0);

            if (IsPointInsidePolyline(mid, boundary))
            {
                AddUniquePoint(allInternalPoints, edgePoints[j]);
                AddUniquePoint(allInternalPoints, edgePoints[j + 1]);
            }
        }
    }

    if (allInternalPoints.Count < 3) return new List<Polyline>();
}
```

凸包重建阶段：使用 Andrew 凸包算法（单调链）对收集到的内部点进行排序，生成顺时针/逆时针有序的顶点序列，创建闭合的裁剪多边形。

凸包算法优势：

- 能处理大多数凸多边形场景
- 消除手动排序导致的乱连线
- 经典算法实现，具有理论支撑

```
// 2. 使用凸包算法重新排序顶点，消除乱连线
List<Point3d> sortedPoints = ComputeConvexHull(allInternalPoints);

// 3. 生成最终的多边形
return new List<Polyline> { CreateCleanPolyline(sortedPoints, tr, db) };
}

/// <summary>
/// Andrew 凸包算法（单调链），确保顶点按逆时针顺序排列。
/// </summary>
/// <param name="points"></param>
/// <returns></returns>
1 个引用 | Gpt
private static List<Point3d> ComputeConvexHull(List<Point3d> points)
{
    // 先按 X 再按 Y 排序
    var sorted = points.OrderBy(p => p.X).ThenBy(p => p.Y).ToList();
    if (sorted.Count <= 2) return new List<Point3d>(sorted);

    List<Point3d> hull = new List<Point3d>();

    // 构建下半部分
    foreach (var p in sorted)
    {
        while (hull.Count >= 2 && Cross(hull[hull.Count - 2], hull[hull.Count - 1], p) <= 0)
            hull.RemoveAt(hull.Count - 1);
        hull.Add(p);
    }

    // 构建上半部分
    int lowerCount = hull.Count;
    for (int i = sorted.Count - 2; i >= 0; i--)
    {
        Point3d p = sorted[i];
        while (hull.Count > lowerCount && Cross(hull[hull.Count - 2], hull[hull.Count - 1], p) <= 0)
            hull.RemoveAt(hull.Count - 1);
        hull.Add(p);
    }

    // 移除最后一个重复的起点
    if (hull.Count > 0)
        hull.RemoveAt(hull.Count - 1);

    return hull;
}
```

```

/// <summary>
/// 计算向量 O->A 与 O->B 的叉积（二维），用于判断旋转方向。
/// 正值表示 O->B 在 O->A 的逆时针方向（保留点）。
/// </summary>
/// <param name="o"></param>
/// <param name="a"></param>
/// <param name="b"></param>
/// <returns></returns>
2 个引用 | Gpt
private static double Cross(Point3d o, Point3d a, Point3d b)
{
    return (a.X - o.X) * (b.Y - o.Y) - (a.Y - o.Y) * (b.X - o.X);
}

/// <summary>
/// 创建闭合的多边形
/// </summary>
/// <param name="points"></param>
/// <param name="tr"></param>
/// <param name="db"></param>
/// <returns></returns>
1 个引用 | Gpt
private static Polyline CreateCleanPolyline(List<Point3d> points, Transaction tr, Database db)
{
    if (points.Count < 3) return null;

    Polyline pline = new Polyline(points.Count);
    for (int i = 0; i < points.Count; i++)
        pline.AddVertexAt(i, new Point2d(points[i].X, points[i].Y), 0, 0, 0);
    pline.Closed = true;
    pline.Layer = "填充种子后";

    BlockTable bt = (BlockTable)tr.GetObject(db.BlockTableId, OpenMode.ForRead);
    BlockTableRecord btr = (BlockTableRecord)tr.GetObject(bt[BlockTableRecord.ModelSpace], OpenMode.ForWrite);
    btr.AppendEntity(pline);
    tr.AddNewlyCreatedDBObject(pline, true);
    return pline;
}

/// <summary>
/// 向列表中添加不重复的点（容差去重）
/// </summary>
/// <param name="list"></param>
/// <param name="pt"></param>
2 个引用 | Gpt
private static void AddUniquePoint(List<Point3d> list, Point3d pt)
{
    foreach (Point3d existing in list)
    {
        if (pt.DistanceTo(existing) < Tolerance.Global.EqualPoint) return;
    }
    list.Add(pt);
}
}

```

局限性：

- 对复杂凹多边形可能将凹角包进去，产生微小的边界变形
- 这是手工算法在无原生布尔运算支持下的工程权衡

七、统计功能

Statistics 命令通过遍历指定边界的全部瓦片列表，按 IsClipped 字段分类统计：

- 未裁剪种子：个数及总面积
- 裁切种子：个数及总面积

使用 Dictionary<ObjectId, List<Tile>>确保每个边界都有独立的统计数据，支持多次填充、多次统计。

八、图层管理

- 生成瓦片统一放入"填充种子后"图层
- 边界位于"边界"图层
- 种子位于"种子"图层
- 填充完成后调用 `SetLayerOrder` 调整绘图次序，确保边界在瓦片之上

```
/// <summary>
/// 设置图层的顺序
/// </summary>
/// <param name="topLayerName"></param>
/// <param name="bottomLayerName"></param>
/// <param name="boundaryId"></param>
1 个引用 | Gpt
private static void SetLayerOrder(string topLayerName, string bottomLayerName, ObjectId boundaryId)
{
    Database db = HostApplicationServices.WorkingDatabase;
    using (Transaction tr = db.TransactionManager.StartTransaction())
    {
        LayerTable layerTable = (LayerTable)tr.GetObject(db.LayerTableId, OpenMode.ForRead);
        if (!layerTable.Has(topLayerName) || !layerTable.Has(bottomLayerName))
        {
            tr.Commit();
            return;
        }

        BlockTable bt = (BlockTable)tr.GetObject(db.BlockTableId, OpenMode.ForRead);
        BlockTableRecord btr = (BlockTableRecord)tr.GetObject(bt[BlockTableRecord.ModelSpace], OpenMode.ForWrite);
        DrawOrderTable drawOrder = (DrawOrderTable)tr.GetObject(btr.DrawOrderTableId, OpenMode.ForWrite);

        ObjectIdCollection topIds = new ObjectIdCollection();
        ObjectIdCollection bottomIds = new ObjectIdCollection();
        foreach (ObjectId objId in btr)
        {
            Entity ent = tr.GetObject(objId, OpenMode.ForRead) as Entity;
            if (ent == null) continue;
            if (ent.Layer == topLayerName) topIds.Add(objId);
            else if (ent.Layer == bottomLayerName) bottomIds.Add(objId);
        }
        if (!topIds.Contains(boundaryId))
        {
            Entity ent = tr.GetObject(boundaryId, OpenMode.ForRead) as Entity;
            if (ent != null && ent.Layer == topLayerName) topIds.Add(boundaryId);
        }

        if (topIds.Count > 0 && bottomIds.Count > 0)
        {
            drawOrder.MoveToBottom(bottomIds);
            drawOrder.MoveToTop(topIds);
        }
        tr.Commit();
    }
}
```

九、关键技术难点与解决

难点	解决方案
旋转种子下网格去重	放弃 BFS，改用行列遍历
行列范围计算	8 采样点投影取整，扩一格保证覆盖
凹角误判为"完全在内"	IsFullyInside 增加边相交检测
圆弧边界点包含判断	射线法 + 圆弧离散化为 10 段直线
裁剪多边形乱连线	边修剪 + Andrew 凸包算法
多次填充统计	Dictionary<ObjectId, List<Tile>>按边界 ID 独立存储

十、总结与展望

本插件实现了一个完整的 CAD 种子填充工具，核心算法包括旋转网格全覆盖、精确的内外判断、基于凸包的裁剪重建。虽然裁剪算法在某些复杂凹多边形下尚有优化空间，但整体架构清晰、逻辑严密，充分展示了算法设计能力、工程实现能力和问题解决能力。

后续可在以下方向进行优化：

1. 引入 `MPolygon` 原生布尔运算，彻底解决凹多边形裁剪问题
2. 支持非矩形种子（如圆形、六边形）
3. 增加可视化 UI 界面，优化用户交互体验